Université Mohammed V de Rabat
École Nationale Supérieure d'Informatique et d'Analyse des Systèmes
Département Ingénierie des Systèmes Embarqués
Filière Smart Systems Engineering

TP 5 : Reconnaître le matériau : entraîner, optimiser et arbitrer trois modèles pour la Portenta H7

Projet EcoTri 4.0, sous-projet SP5, jalon J5 « rapport d'optimisation »

Pr.Hassan BERBIA/Pr.Aissam BERRAHOU

2 novembre 2026

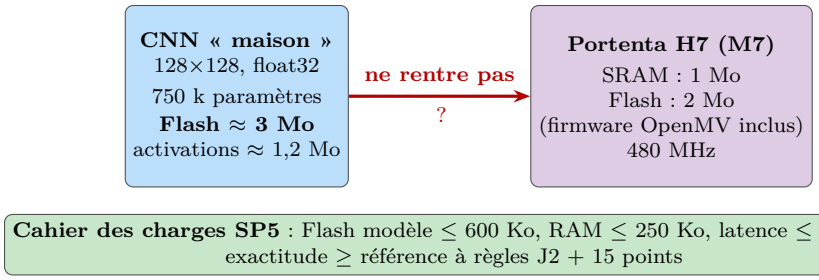
Objectifs du TP

Votre équipe doit livrer le classifieur de matériaux de la station EcoTri 4.0 et **défendre** son choix. Vous construirez un jeu de données mêlant TrashNet et images de la station, entraînerez trois modèles (CNN simple, MobileNetV2 $\alpha = 0{,}35$ en float32, le même en INT8), les mesurerez sur la Portenta, tracerez leur front de Pareto à côté de la référence à règles J2, puis rédigerez le rapport d'optimisation.
À la fin de ce TP, vous serez capable de :

- Construire un jeu d'images équilibré et le découper par session sans fuite.
- Entraîner un CNN depuis zéro et un MobileNetV2 par transfer learning, et calculer leurs coûts.
- Quantifier en INT8, mesurer Flash, RAM et latence sur la cible, et analyser les matrices de confusion.
- Lire un front de Pareto et argumenter un compromis sous contraintes.

Durée : 1 h 45 en séance + finalisation du rapport. **Organisation :** équipes de 3–4, rôles tournants (données, entraînement, mesures sur carte, rapport).
Livrable (J5) : rapport de 4 pages max. + fichiers `.tflite` + notebook + tableau de mesures.

1 Contexte et cahier des charges



Le problème déclencheur : un premier CNN de 3 Mo ne tient pas dans la Portenta.

Exigence	Seuil	Mesure
E1 Flash du modèle	$\leq$ 600 Ko	taille du `.tflite` / rapport EON
E2 RAM d'inférence	$\leq$ 250 Ko	`gc.mem_free()` ou estimation EON
E3 Latence	$\leq$ 100 ms	médiane sur la carte (Tutorial 5)
E4 Exactitude équilibrée	$\geq$ J2 + 15 points	base de test « station »
E5 Rappel du verre	$\geq$ 85 %	base de test « station »

Cahier des charges SP5. E4 et E5 se mesurent sur des images de la station jamais vues à l'entraînement.

2 Partie A : les données (25 min)

1. Téléchargez TrashNet et ne gardez que `cardboard`, `glass`, `metal`, `plastic` (environ 400 à 500 images par classe).
2. Avec `ml_coherence.py` du Tutorial 5, capturez sur la station **au moins 40 images par classe**, réparties en **4 sessions** (session = position de caméra, éclairage, moment). Nommez les fichiers `classe_sSESSION_idx.jpg`.
3. Découpage : TrashNet + sessions 1–2 → entraînement ; session 3 → validation ; session 4 → **test** (jamais regardée avant la partie E).

```
# donnees.py - découpage par session, sans fuite (chapitre 4)
import pathlib, shutil, re
src, dst = pathlib.Path("station"), pathlib.Path("ecotri_img")
for f in src.glob("*.jpg"):
    classe, session = re.match(r"(\w+)_s(\d)_", f.name).groups()
    part = {"1": "train", "2": "train", "3": "val", "4": "test"}[session]
    (dst / part / classe).mkdir(parents=True, exist_ok=True)
    shutil.copy(f, dst / part / classe / f.name)
# TrashNet ne va QUE dans train (copie des 4 dossiers renommés en carton/verre/metal/
    plastique)
```

Question

Pourquoi TrashNet ne doit-il pas figurer dans la base de test ? Pourquoi faut-il enregistrer les quatre classes dans **chaque** session ? Notez le nombre d'images par classe et par partie dans le rapport.

3 Partie B : trois modèles (35 min)

B1. Calculer avant d'entraîner

Complétez le tableau à la main (formules du cours), puis vérifiez avec `model.summary()` et `python3 gen_images.py`.

Modèle (96×96×3, 4 classes)	Paramètres	MACs	Poids INT8	Pic activ. INT8
M1 : CNN simple (3 conv 16/32/64, GAP, dense)				
M2 : MobileNetV2 $\alpha = 0{,}35$ float32			×4 =	×4 =
M3 : MobileNetV2 $\alpha = 0{,}35$ INT8				

B2. Entraîner

```
# modeles.py - M1 (depuis zéro) et M2 (transfer learning) sur les mêmes données
import tensorflow as tf
T = 96
lire = lambda p, sh: tf.keras.utils.image_dataset_from_directory(p, image_size=(T, T)
    ,
                                     batch_size=32, shuffle=sh, label_mode="int")
train, val, test = lire("ecotri_img/train", True), lire("ecotri_img/val", False),
    lire("ecotri_img/test", False)
aug = tf.keras.Sequential([tf.keras.layers.RandomFlip("horizontal"), tf.keras.layers.
    RandomRotation(0.15),
                           tf.keras.layers.RandomBrightness(0.25), tf.keras.layers.
    RandomContrast(0.2)])
L = tf.keras.layers
def m1():
    i = tf.keras.Input((T, T, 3)); x = L.Rescaling(1 / 255.)(aug(i))
    for f in (16, 32, 64):
        x = L.Conv2D(f, 3, strides=2, padding="same", activation="relu")(x)
        x = L.BatchNormalization()(x)
    x = L.GlobalAveragePooling2D()(x); x = L.Dropout(0.3)(x)
    return tf.keras.Model(i, L.Dense(4, activation="softmax")(x))
def m2():
    b = tf.keras.applications.MobileNetV2((T, T, 3), alpha=0.35, include_top=False,
    weights="imagenet")
    b.trainable = False
    i = tf.keras.Input((T, T, 3)); x = L.Rescaling(1 / 127.5, offset=-1)(aug(i))
    x = L.GlobalAveragePooling2D()(b(x, training=False)); x = L.Dropout(0.2)(x)
    return tf.keras.Model(i, L.Dense(4, activation="softmax")(x)), b
cb = [tf.keras.callbacks.EarlyStopping(patience=5, restore_best_weights=True)]
M1 = m1(); M1.compile("adam", "sparse_categorical_crossentropy", ["accuracy"])
M1.fit(train, validation_data=val, epochs=40, callbacks=cb)
M2, base = m2(); M2.compile("adam", "sparse_categorical_crossentropy", ["accuracy"])
M2.fit(train, validation_data=val, epochs=15, callbacks=cb)
base.trainable = True                                    # fine-tuning des 20 dernières
    couches
for l in base.layers[:-20]: l.trainable = False
M2.compile(tf.keras.optimizers.Adam(1e-4), "sparse_categorical_crossentropy", ["
    accuracy"])
M2.fit(train, validation_data=val, epochs=10, callbacks=cb)
M1.save("m1.keras"); M2.save("m2.keras")
```

B3. Quantifier

Convertissez M1 et M2 en INT8 avec la fonction de l'étape 1 du Tutorial 5 (calibration sur 200 images d'entraînement, **dont des images de la station**). Vous obtenez M1-INT8 et M3 = M2-INT8.

Astuce

Variante sans code : créez un projet Edge Impulse « Image classification », bloc **Transfer learning (MobileNetV2 96×96 0.35)**, et comparez dans l'onglet **Deployment** les estimations float32 / quantized (int8) avec et sans **EON Compiler**. Reportez ces estimations de RAM et de Flash dans votre tableau.

4 Partie C : mesures sur la cible (20 min)

Pour chaque modèle (M1 float32, M1-INT8, M2, M3), utilisez `ml_charger.py` et `ml_classer.py` du Tutorial 5.

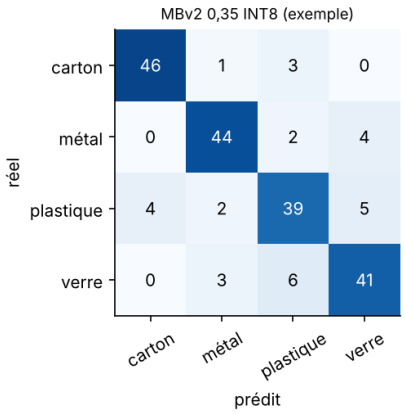
Modèle	Fichier (Ko)	RAM (Ko)	Latence méd. (ms)	FPS	Charge ?	E1–E3
Règles J2	–				oui	
M1 float32						
M1 INT8						
M2 float32						
M3 INT8						

Question

Calculez pour chaque modèle le débit effectif en MACs par cycle. Les modèles INT8 l'améliorent-ils ? Que conclure sur la relation entre MACs et latence ?

5 Partie D : exactitude et confusions (15 min)

```
# evaluer.py - exactitude équilibrée, rappel par classe et matrice de confusion (base test station)
import numpy as np, tensorflow as tf
from sklearn.metrics import confusion_matrix, balanced_accuracy_score, classification_report
def evaluer_tflite(fichier, ds):
    it = tf.lite.Interpreter(fichier); it.allocate_tensors()
    e, s = it.get_input_details()[0], it.get_output_details()[0]
    y, p = [], []
    for x, t in ds.unbatch():
        x = x.numpy()[None]
        if e["dtype"] == np.int8:
            se, ze = e["quantization"]; x = np.clip(np.round(x / se + ze), -128, 127).astype(np.int8)
        it.set_tensor(e["index"], x.astype(e["dtype"])); it.invoke()
        p.append(int(np.argmax(it.get_tensor(s["index"])))); y.append(int(t))
    print(fichier, "exactitude équilibrée = %.3f" % balanced_accuracy_score(y, p))
    print(classification_report(y, p, digits=3)); print(confusion_matrix(y, p))
for f in ["m1_f32.tflite", "m1_int8.tflite", "m2_f32.tflite", "m3_int8.tflite"]:
    evaluer_tflite(f, test)
```



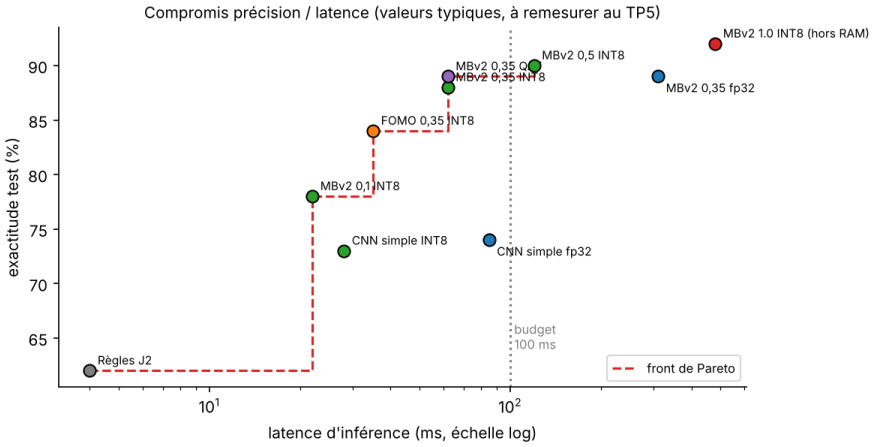
Exemple de matrice attendue. Commentez les vôtres : quelles confusions, quelle explication physique ?

Question

Évaluez aussi le classifieur à règles J2 sur les mêmes images de test (session 4). La perte due à l'INT8 est-elle la même pour M1 et pour M3 ? Lequel est le plus sensible, et pourquoi (indice : nombre de paramètres, BatchNorm, plages d'activations) ?

6 Partie E : front de Pareto et décision (10 min)

```
# pareto.py - à compléter avec VOS mesures
import matplotlib.pyplot as plt
mes = {"Règles J2": (4, 0.61), "M1 f32": (0, 0), "M1 INT8": (0, 0),
       "M2 f32": (0, 0), "M3 INT8": (0, 0)}            # (latence ms, exactitude é
    quilibrée)
pts = sorted(mes.items(), key=lambda kv: kv[1][0]); front, best = [], -1
for n, (l, a) in pts:
    if a > best: front.append((l, a)); best = a
for n, (l, a) in mes.items():
    plt.scatter(l, a, s=50); plt.annotate(n, (l, a), xytext=(4, 4), textcoords="
    offset points")
plt.step(*zip(*front), where="post", ls="--", color="red", label="front de Pareto")
plt.axvline(100, ls=":", color="gray"); plt.xscale("log")
plt.xlabel("latence sur la Portenta (ms)"); plt.ylabel("exactitude équilibrée (test
    station)")
plt.legend(); plt.savefig("pareto_equipe.png", dpi=150)
```



Allure attendue (valeurs typiques du cours). Votre figure doit contenir **vos** mesures.

7 Jalon J5 : rapport d'optimisation

1. **Contraintes** E1–E5 et leur justification (budget partagé avec le modèle audio).
2. **Données** : sources, effectifs par classe et par partie, découpage par session.
3. **Modèles** : architectures, coûts calculés (B1) et mesurés (C).
4. **Résultats** : tableau complet, matrices de confusion, front de Pareto avec la référence J2.
5. **Décision** : modèle retenu, exigences satisfaites ou non, seuil de confiance proposé.
6. **Limites et suite** : décalage de domaine, QAT, distillation, FOMO, intégration en lecture 6.

Critère	Points	Attendu
Fonctionnement sur la carte	4	M3 classe en temps réel, décision JSON
Qualité des mesures	5	Flash, RAM, latence, exactitude, mesurées et reproductibles
Rigueur de l'évaluation	4	découpage par session, test station, comparaison J2
Justification du choix	5	Pareto, contraintes, coût des erreurs (verre)
Qualité du code et du rapport	2	scripts commentés, figures lisibles
Total	**20**	

Grille critériée du jalon J5.

8 Pour aller plus loin (bonus)

— **QAT** : `tfmot.quantization.keras.quantize_model(M2)` puis 5 époques ; la perte INT8 diminue-t-elle ?

— **Distillation** : M2 (ou un MobileNetV2 1.0 sur PC) comme enseignant de M1, $T = 4$, $\alpha = 0{,}3$; gain sur M1-INT8 ?

— **Élagage structuré** : réduisez de moitié les filtres de M1 ; comparez à l'élagage non structuré de `tfmot` (50 %) en taille, latence et exactitude.

— **Largeur et résolution** : MobileNetV2 $\alpha = 0{,}1$ et $0{,}5$; résolution 64 et 128. Complétez le front de Pareto.

— Références : MIT 6.5940 (S. Han), labs Pruning et Quantization ; Harvard CS249r, chapitres « Model Optimizations » et « Benchmarking AI » (mlsysbook.ai).